Constants in C++

Manifest constants are defined by #define
Manifest constants are replaced by CPP (C Pre-Processor)

```
#include <iostream>
#include <cmath>
#define TWO 2
#define PI 4.0*atan(1.0)
using namespace std;
int main() {
 int r = 10;
double peri = TWO * PI * r;
cout << "Perimeter = " << peri << endl;
return 0; }
```

• TWO is a manifest constant • CPP replaces the token TWO by 2 • PI is a manifest constant • CPP replaces the token PI by 4.0*atan(1.0) • TWO & PI look like variables • Compiler sees them as constants

Using const

```
#include <iostream>
#include <cmath>
using namespace std;
const int TWO = 2; const double PI = 4.0*atan(1.0);
int main() {
int r = 10;
double peri = TWO * PI * r; // No replacement by CPP
cout << "Perimeter = " << peri << endl;
return 0; }
```

Output: Perimeter = 314.159 Perimeter = 314.159

TWO is a const variable initialized to 2

PI is a const variable initialized to 4.0*atan(1.0)

TWO & PI are variables

Type of TWO is const int

Type of PI is const double

Differences between manifest const and const variable

Prefer const over #define

Manifest Constant

- Is not type safe
- Replaced textually by CPP
- Cannot be watched in debugger
- Evaluated as many times as replaced

Constant Variable

- Has its type
- Visible to the compiler
- Can be watched in debugger
- Evaluated only on initialization

Const and Pointers

const-ness can be used with Pointers in one of the two ways:
Pointer to Constant data where the pointee (pointed data) cannot be changed
Constant Pointer where the pointer (address) cannot be changed
Consider usual pointer-pointee computation (without const):

```
int m = 4; int n = 5;
int * p = &n; // p points to n. *p is 5
...
n = 6; // n and *p are 6 now
*p = 7; // n and *p are 7 now.  POINTEE changes
...
p = &m; // p points to m. *p is 4. POINTER changes
*p = 8; // m and *p are 8 now. n is 7. POINTEE changes
```

Consider pointed data

```
int m = 4;
 const int n = 5;
 const int * p = &n;
 ... n = 6; // Error: n is constant and cannot be changed
 *p = 7; // Error: p points to a constant data (n) that cannot be changed
p = &m; // Okay
*p = 8; // Okay
```

Interestingly,

```
int n = 5;
const int * p = &n;
 ... n = 6; // Okay
*p = 6; // Error: p points to a 'constant' data (n) that cannot be
    changed
```

Consider pointer

```
int m = 4, n = 5;
int * const p = &n;
 ... n = 6; // Okay
*p = 7; // Okay. Both n and *p are 7 now
 ... p = &m; // Error: p is a constant pointer and cannot be changed
```

By extension, both can be const

```
const int m = 4; const int n = 5;
const int * const p = &n; ...
n = 6; // Error: n is constant and cannot be changed
*p = 7; // Error: p points to a 'constant' data (n) that cannot be changed
 ... p = &m; // Error: p is a constant pointer and cannot be changed
```

Finally, to decide on const-ness, draw a mental line through *

```
int n = 5; int * p = &n; // non-const-Pointer to non-const-Pointee
const int * p = &n; // non-const-Pointer to const-Pointee
 int * const p = &n; // const-Pointer to non-const-Pointee
const int * const p = &n; // const-Pointer to const-Pointee
```

Macros and Inline functions

Macros with Parameters are defined by #define

Macros with Parameters are replaced by CPP

Source Program Program after CPP

```
#include <iostream> using namespace std;
#define SQUARE(x) x * x
int main() { int a = 3, b;
b = SQUARE(a);
cout << "Square = " << b << endl; return 0;
}
```

```
int main() { int a = 3, b;
b = a * a; // Replaced by CPP
cout << "Square = " << b << endl; return 0;
}
Square = 9
```

SQUARE(x) is a macro with one param • CPP replaces the SQUARE(x) substituting x with a • SQUARE(x) looks like a function • Compiler does not see it as function

Consider the example:

```
#include <iostream> using namespace std;
#define SQUARE(x) x * x
int main() { int a = 3, b;
b = SQUARE(a + 1); // Wrong macro expansion
cout << "Square = " << b << endl;
return 0;
}
```

Output is 7 in stead of 16 as expected. On the expansion line it gets:

```
b = a + 1 * a + 1;
```

To fix:

```
#define SQUARE(x) (x) * (x)
```

Now:

```
b = (a + 1) * (a + 1);
```

```
#include <iostream>
using namespace std;
#define SQUARE(x) (x) * (x)
int main() {
 int a = 3, b;
b = SQUARE(++a);
cout << "Square = " << b << endl;
return 0;
}
```

Output is 25 in stead of 16 as expected. On the expansion line it gets:
b = (++a) * (++a);
and a is incremented twice before being used! There is no easy fix.

- An inline function is just like other functions
- The function prototype is preceded by the keyword inline
- An inline function is expanded (inlined) at the site of its call and the overhead of passing parameters between caller and callee (or called) functions is avoided

Define the function Prefix function header with inline
Compile function body and function call together

```
#include <iostream>
using namespace std;
inline int SQUARE(int x) { return x * x; }
 int main() {
 int a = 3, b;
 b = SQUARE(a);
 cout << "Square = " << b << endl;
return 0; }
```

Square =9

SQUARE(x) is a function with one param
inline SQUARE(x) is equally efficient
SQUARE(a + 1) fails with macros and inline SQUARE(a + 1) works
SQUARE(++a) fails with macros and inline SQUARE(++a) works

Macros

- Expanded at the place of calls • •
- Efficient in execution
- Code bloats
- Has syntactic and semantic pitfalls
- Type checking for parameters is not done
- Errors are not checked during compilation
- Not available to debugger

Inline Functions

- Expanded at the place of calls
- Efficient in execution
- Code bloats
- No pitfall
- Type checking for parameters is robust
- Errors are checked during compilation
- Available to debugger in DEBUG build

Limitations of Inline Functions

- inlineing is a directive – compiler may not inline functions with large body
- inline functions may not be recursive
- Function body is needed for inlineing at the time of function call. Hence, implementation hiding is not possible.

Reference Variable

- A reference is an alias / synonym for an existing variable
- int i = 15; // i is a variable int &j = i; // j is a reference to i
- i ← variable 15 ← memory content 200 ← address j ← alias or reference

```
#include <iostream>
using namespace std;
int main() {
 int a = 10, &b = a; // b is reference of a // a and b have the same memory
cout << "a = " << a << ", b = " << b << endl;
cout << "&a = " << &a << ", &b = " << &b << endl;
++a; // Changing a appears as change in b
cout << "a = " << a << ", b = " << b << endl;
++b; // Changing b also changes a
cout << "a = " << a << ", b = " << b << endl;
return 0;
}
```

```
a = 10, b = 10
&a = 002BF944, &b = 002BF944
a = 11, b = 11 a = 12, b = 12
```

- a and b have the same memory location and hence the same value
- Changing one changes the other and vice-versa

```
#include <iostream>
using namespace std;
void Function_under_param_test(// Function prototype int &b, // Reference parameter int c); //
    Value parameter
int main() {
 int a = 20;
 cout << "a = " << a << ", &a = " << &a << endl << endl;
 Function_under_param_test(a, a); // Function call
return 0;
}
void Function_under_param_test(int &b, int c) { // Function definition
cout << "b = " << b << ", &b = " << &b << endl << endl;
 cout << "c = " << c << ", &c = " << &c << endl << endl; }
```

------- Output ------a = 20, &a = 0023FA30 b = 20, &b = 0023FA30 c = 20, &c = 0023F95C

Param b is call-by-reference while param c is call-by-value

Actual param a and formal param b get the same value in called function

Actual param a and formal param c get the same value in called function

However, actual param a and formal param c have different addresses in called function

SWAP in C

Call-by-value:

```
#include <stdio.h>
void swap(int, int); // Call-by-value
int main() {
int a = 10, b = 15;
printf("a= %d & b= %d to swap\n", a, b);
swap(a, b);
printf("a= %d & b= %d on swap\n", a, b); return 0; }
void swap(int c, int d){ int t; t = c; c = d; d = t; }
```

Call-by-address:

```
#include <stdio.h>
void swap(int *, int *); // Call-by-address
 int main() { int a=10, b=15;
printf("a= %d & b= %d to swap\n", a, b);
swap(&a, &b);
printf("a= %d & b= %d on swap\n", a, b);
return 0; }
void swap(int *x, int *y){ int t; t = *x; *x = *y; *y = t; }
```

```
#include <iostream>
using namespace std;
void swap(int&, int&); // Call-by-reference
int main() { int a = 10, b = 15;
cout<<"a= "<<a<<" & b= "<<b<<"to swap"<<endl;
swap(a, b); cout<<"a= "<<a<<" & b= "<<b<<"on
   swap"<<endl;
return 0; }
void swap(int &x, int &y) { int t ; t = x ; x = y ; y = t ; }
```

Reference parameter as const

A reference parameter may get changed in the called function
Use const to stop reference parameter being changed
const reference

```
#include <iostream> using namespace std;
int Ref_const(const int &x) { ++x; // Not allowed return (x); }
int main() { int a = 10, b;
b = Ref_const(a);
cout << "a = " << a <<" and" << " b = " << b; return 0; }
```

```
#include <iostream> using namespace std;
int Ref_const(const int &x) {
return (x + 1);
}
int main() {
int a = 10, b; b = Ref_const(a);
 cout << "a = " << a << " and" << " b = " << b; return 0; }
```

Return by Reference

A function can return a value by reference

```
#include <iostream>
 using namespace std;
int Function_Return_By_Val(int &x) {
 cout <<"x = "<<x<<" &x = "<<&x<<endl;
 return (x); }
int main() {
int a = 10;
 cout <<"a = "<<a<<" &a = "<<&a<<endl;
const int& b = Function_Return_By_Val(a);
cout <<"b = "<<b<<" &b = "<<&b<<endl;
return 0;
}
#include <iostream>
using namespace std;
int& Function_Return_By_Ref(int &x)
 { cout <<"x = "<<x<<" &x = "<<&x<<endl;
 return (x); }
 int main() {
 int a = 10; cout <<"a = "<<a<<" &a = "<<&a<<endl;
const int& b = // const optional Function_Return_By_Ref(a);
cout <<"b = "<<b<<" &b = "<<&b<<endl; return 0;
}
```

Difference between pointers and references

Pointers

- Refers to an address
- Pointers can point to NULL. int *p = NULL; // p is not pointing
- Pointers can point to different variables at different times, int a, b, *p; p = &a; // p points to a ... p = &b // p points to b
- Allows users to operate on the address – diff pointers, increment, etc.
- Array of pointers can be defined

References

- Refers to an address
- References cannot be NULL int &j ; //wrong
- For a reference, its referent is fixed int a, c, &b = a; // Okay ..... &b = c // Error
- Does not allow users to operate on the address. All operations are interpreted for the referent
- Array of references not allowed

Dynamic Memory Management

- Memory management in C malloc() & free()
  Memory management in C++ new and delete
  Array new[] and delete[]

C Program

```
#include <stdio.h> #include <stdlib.h>
int main() { int *p = (int *)malloc(sizeof(int)); *p = 5;
printf("%d", *p);
free(p);
return 0;
} ----5
```

C++ Program

```
#include <iostream> #include <cstdlib> using namespace std;
int main() { int *p = (int *)malloc(sizeof(int)); *p = 5;
cout << *p;
free(p);
return 0;
} ----5
```

Dynamic memory management functions in stdlib.h header for C (cstdlib header for C++) malloc() allocates the memory on heap sizeof(int) needs to be provided Pointer to allocated memory returned as void * – needs cast to int * Allocated memory is released by free() from heap calloc() and realloc() also available in both languages

C++ introduces operators new and delete to dynamically allocate and de-allocate memory:

```
#include <iostream>
using namespace std;
int main() { int *p = new int(5);
cout << *p;
delete p;
return 0;
} ----5
```

- Operator new for allocation on heap
- No size specification needed, type suffices
- Allocated memory returned as int *
- No casting needed
- Can be initialized
- Operator delete for de-allocation from heap
- Core language feature – no header needed

Dynamically managed Arrays

malloc() & free()

```
#include <iostream> #include <cstdlib> using namespace std;
int main() { int *a = (int *)malloc(sizeof(int)* 3); a[0] = 10; a[1] = 20; a[2] = 30;
for (int i = 0; i < 3; ++i) cout << "a[" << i << "] = " << a[i] << " "; cout << endl;
free(a);
return 0;
} ----a[0] = 10 a[1] = 20 a[2] = 30
```

new[] & delete[]

```
#include <iostream> using namespace std;
int main() { int *a = new int[3];
a[0] = 10; a[1] = 20; a[2] = 30;
for (int i = 0; i < 3; ++i)
cout << "a[" << i << "] = " << a[i] << " "; cout << endl;
delete [] a;
return 0;
} ----a[0] = 10 a[1] = 20 a[2] = 30
```

- Allocation and De-Allocation must correctly match. Do not free the space created by new using free(). And do not use delete if memory is allocated through malloc(). These may results in memory corruption